1. Linux Fundamentals

Questions Yet to be Solved :

1. can we delete hidden dir "." and ".." ?
2. ".." dir is also present in / partition, why cannot we go further back?
3. Why "+" is not changed back to "." in "ls -l", when we removed all apecific user/groups from acl? (b/c + shows acl applied??)
4. consols inside runlevel 2, 3, 4 are perhaps 6, but their drivers? tty files inside /dev/ are upto 63 ???
5. consols shifting is not perfect....some time works sometime it doesn't ?
6. symbols / operators / wild characters i.e., (), {}, [], <>, etc
7. How many critical files are there in linux?
8. How to take backups into remote machine / SAN / NAS / LTO / Tape drive?
9. Full procedure of LUN disk scanning (detailed)?
10. When we delete a partition (from in between partitions), a partition is shifted ?

1.1 Basic command lines

Revise all basic command lines from bopinder's lectures, much stuff of RHEL intensive and other sources is also added to these notes.

If we do not know about a certain command, we can check as;

```
whatis pwd  # give short description of pwd command
man pwd     # opens manual for pwd command (:q to exit)
```

Command	Fx
z	write "z" and press TAB TAB, it will list all commands starting from "z"
`stat abc.txt`	will give all details of abc.txt
bc	opens linux calculator
Ctrl Shift +	to increase font size
Ctrl -	to decrease font size
shift+g	to go to the end of file in editor
Alt+TAB and then keep holding Alt and using TAB or arrow keys	to nagigate amongst opened windows in "windows OS"
press and hold Alt+TAB and use arrow keys	to navigate amongst opened windows of genome terminals in "linux OS"

1.2 File System Hirarchy

1.2.1 `/` | **root** | parent directory

It contains many system directories, i.e.,

1.2.1.1 /boot

Its main function is to load OS over RAM when we turn on machine.
It contains files required and used by machine while turning on or rebooting. It varifies if all hardwares are properly connented and working, if else, it give us a prompt msg that your specific driver is missing or else, etc.

It contains many files, most important of them are "kernel" and "GRUB".

- "kernel" file | `Vmlinuz...86_64`. `du -sh Vmlinuz.....86_64` = almost 4MB
    - Here in this command:
        - `du`= disk usage
        - `s` = summarize output
        - `h` = print size in human readible format

Kernel is the father of `fdisk`

- Second most important dir inside `/boot` is `grub/`

In linux "GRUB" and "LILO" are two boot loaders.

1.2.1.2 /dev

- It contains driver files.
- There are also a lot many special files inside `/dev`
    - we can see these files using :
    - `ls -l /dev/* | grep ^c` = list character files
    - `ls -l /dev/* | grep ^s` = list socket files

Charachter file vs Normal(text) file | difference

Normale File	Character File
"/dev/abc.txt"	`/dev/lp0` or `/dev/sda`
normal file	special file
used for data writing	helps in connecting two components
normal text file	a driver file
in human readible	in binary
non executable	executable
written in HDD	loads in RAM (not in HDD)

> Keep in mind:

- `/dev/sda` = disk driver file (driver binary file)
- `/dev/lp0` = printer driver file (driver binary file)
- `/sbin/fdisk` = also deals with drivers (it is command binary for us to use)
- `/dev/nst0` = tape device driver | LTO (linear tape open)
    - tape devices are actually used to store backups
    - LTOs have CC (caset cases)
    - NW card and FC
    - Servers store their backups in LTOs and their drivers in servers are named as `nst0`, `nst1`, etc
    - tape_concept
- `/dev/loop0` = driver for iso images
    - these are by default in total 7 in no. (`loop0` to `loop6`)
- `/dev/null` = nullify all inputs
    - just like if we run `man ls > abc`= "ls manual" saves in abc
    - but if we run `man ls > /dev/null` = nullify the output, do not show "ls manual"

How to take Backup in TAPE

Remember one or twp model numbers of lto.

Normally we take backup like:

`tar -cvf /mnt/box.tar /home/*` = taking backup of entire `/home/`

For tape drive, we will just change destination point, accordingly:

- `tar -cvf /dev/nst0 /home/*`
- `tar -cvf /dev/nst0 /*` = of complete server

How to manage TAPE

We can manage it using `mt` (manage tape)
It can be installed using `yum` !!??
Write 15 examples (and concept) to manage it from this page

- `mt -f /dev/nst0 status` = check tape status
- `mt -f /dev/nst0 forward` = to forward tape header
- `mt -f /dev/nst0 rewind` = to rewind tape header
- `mt -f /dev/nst0 eject` = to remove tape
- `mt -f /dev/nst0 status` = check tape status
- `mt -f /dev/nst0 erase` = to erase

1.2.1.3 /etc

It contains service config. files

1.2.1.4 /usr/

It contains all installed softwares

1.2.1.5 /usr/bin

It contains all command files that can be used by normal user.

1.2.1.6 /usr/sbin

It contains command files that can be used by super user.

> we should keep on taking **snapshots** while doing pngys with root files, b/c if we crashed the machine at any point, we can restore the system state from last point of **snapshot**, just like backup.

1.2.1.7 /home

home dir of normal user.

1.2.1.8 /root

home dir of root user.

1.2.1.9 /lib

It conctains `.so` files (just like ".dll" files in windows)

1.2.1.10 /var | (varies)

It contains log files. (stores all daily activities) | (just like event viewer in windows).
Its size keeps in increasing (varying) and contains system logs inside `/var/log/`.

Some important files of inside `/var/log/` includes:

1. `/var/log/messages`
    - `less /var/log/messages` OR `more /var/log/messages` = to read system common logs
    - `head /var/log/messages` = show top 10 lines (default)
    - `head -5 /var/log/messages` = show top 20 lines
    - `tail /var/log/messages` = show last 10 lines
    - `tail -20 /var/log/messages` = show last 20 lines
    - `tail -f /var/log/messages` = shows real time tail of logs
    - `tailf /var/log/messages` = same as above
2. `/var/log/secure`
    - it contains authentication files inside it

Log Rotation | just overview

thegeekstuff.com | Link For Details

System itself keeps on taking backup of some files of `/var/log/` and saving those backup files within `/var/log/`, it is known as *log rotation*.

- backup file is created, when size of its main file exceeds a certain (defined) size.
- by default, at a time, only 4 backup files of any file are present (as there comes 5th file, oldest one will be deleted, automatically)
- we can change these default settings of log rotation service
    - Example:
        - backups of `/var/log/messages` and `/var/log/secure`, etc

`dmesg` | diagnostic message

`dmesg` is just like device manager in windows

- `dmesg` helps us read hardware logs that are detected by OS
    - commands to read it:
        - `dmesg`
        - `less dmesg`, `more dmesg`
        - `head dmesg`, `head -20 dmesg`
        - `tail dmesg`, `tail -20 dmesg`
        - `tail -f dmesg`, `tailf dmesg`, `tailf -20 dmesg`
- We can search specific driver, out of all here:
    - `dmesg | grep -i eth`= to search for "eth"
- But this `dmesg` do not give detail info of motherboard, for that, we have:
    - `dmidecode | less`
    - `biosdecode | less`

1.2.1.11 /opt

Stores 3rd party application softwares.

1.2.1.12 /mnt

It is actually an optional directory that a user can use as a mountpoint if/when needed.

1.2.1.13 /tmp

It contains temporary files

- .lck files (it remains there till the time application is running)

Sticky Bit Permission (yet to cover)

`/tmp` looks green | sticky bit permission (a type of permission)

1.2.1.14 /lost+found

It is basically used to store data recovered files.

1.2.2 FSH | continue

> Can we create separate partitions for "/dev", "/bin", "/sbin", "/lib", "/etc" ?

- No, we cannot seprate partitions for these dir(s) and mout them.
- It is because, these dir(s) contains system files that are required by OS.
- So, these files should always be available to run OS.
- But if we create seprate partitions and mount them on, for these dir(s), they can also be demounted (in that case) and making those system files unavailable for OS.
- OS cannot work without those system files.

1.3 Ownership and Permissions

1.3.1 Overview

```
[abc@xyz opt]# touch file1
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 root root 0 may 1 02:13 file1
```

Here are total 9 field (of properties) ;

> 1."-" = shows it is a file (if d, it is dir)
> 2. "rwx" = root user / owner permission to read, write, execute
> 2. "r-x" = group permission to read and execute
> 2. "r--" = other users permission to read only file1
> 3. "." = shows "acl" applied
> 4. "1" = actually link count, no. of links to open this file (2 in case of empty/fresh dir/) # incase of dir, it actually depends on no. of sub-dir(if empty, it has 2 b/c of 2 hidden subdir)
> 5. "root" = fist root is owner of file1
> 6. "root" = second root is group name of file1
> 7. "0" = it is the size of file1 in bytes, (usually 6 for empty dir)
> 8. "may 1" = date of creation of file
> 8. "02:13" = time of creation of file
> 9. "file1" = name of the file

About HIDDEN SUBDIR :

> we know each dir that we make, has by-default to subdir as

- .
- ..

```
[abc@xyz test1]# pwd
/root/home/Desktop/test1         # just for example
[abc@xyz test1]# ls -al
.
..
[abc@xyz test1]# cd .
```

```
[abc@xyz test1]#        # . will dir again to pwd
[abc@xyz test1]# cd ..
[abc@xyz Desktop]#      # .. will dir to parent dir (if del, this command will not
work for this specific folder, I tried but it was neither showing contents nor
being del)
```

Let's we have a file with permissions as; (just example)

```
---      # bit OFF
rwx      # bit ON
-r-      # 1bit OFF   2bit ON   3bit OFF


# Let's modify our above example;
---     umar        # null permission for umar
rwx     ali         # full permission for ali
-r-     urwa        # partial permission for urwa


# Let's modify more; (write in binary)
000     umar        # OFF permissions for umar
111     ali         # ON permissions for ali
010     urwa        # only READ permission for urwa
```

Let's consider this example now:

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 root root 0 may 1 02:13 file1
```

Converting the above permissions into 3 portions:

```
rwx     r-x     r--
```

Now, we can write in binary as;

```
111     1-1     100
```

Before going further, first we discuss 9 fields of properties of any file;

```
[abc@xyz opt]# touch file1
[abc@xyz opt]# ls -l
```

```
-rwxr-xr--. 1 root root 0 may 1 02:13 file1

1. file type
2. permissions (root,groups,others)
3. selinux
4. link counts
5. owner name
6. group name
7. size in bytes
8. access time
9. file name
```

There are total 7 types of files:

```
[abc@xyz]# touch file1
[abc@xyz]# ls -l
-rwxr-xr--. 1 root root 0 may 1 02:13 file1

# "-" = regular file
# "d" = dir
# "l" = symlink
# "b" = block file
# "c" = character file
# "s" = socket file
# "p" = pipe device file
```

Size (in bytes) of a file: (7th field property)

```
Size is shown in bytes,
    8 bits = 1 byte
    1024 bytes = 1 kb
    1024 kb = 1 MB
    1024 mb = 1 GB
    1024 Gb = 1 TB
    1024 Tb = 1 PB (peta byte)
    1024 pb = 1 ZB (zeta byte)

size = 0 byte for an empty / fresh file
size = 4 kb for an empty / fresh dir
```

> Whenever we make a user in linux, its group is also created by-default with the same name as of user that we created.

1.3.2 Changing ownerships

command	dx

command	dx
chown	change owner
id user1	giveus detailed identity of user1
cat /etc/passwd	to see users
chgrp	change group
chmod	to change permissions

Changing Owner

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 root root 0 may 1 02:13 file1

[abc@xyz opt]# chown user1 file1    # suppose we already have another user named,
user1
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 root 0 may 1 02:13 file1
```

We can change ownership and group at the same time as:

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 root root 0 may 1 02:13 file1

[abc@xyz opt]# chown user1:user2 file1    # suppose we already had other users
named, user1 and user2
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 user2 0 may 1 02:13 file1
```

Changing group name

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 root 0 may 1 02:13 file1

[abc@xyz opt]# chgrp group1 file1    # if we already another group as group1
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 group1 0 may 1 02:13 file1
```

1.3.3 Changing Permissions (symbolic and octal ways)

Ther are two methods for changing permissions:

1.3.3.1 Symbolic Way

we have three categories of users;

- u = user(owner)
- g = group
- o = others

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 group1 0 may 1 02:13 file1
[abc@xyz opt]# chmod o+rwx file1        (giving rwx to others)
[abc@xyz opt]# ls -l
-rwxr-xrwx. 1 user1 group1 0 may 1 02:13 file1
```

OR

```
[abc@xyz opt]# chmod o-rwx file1        (removing rwx from others)
[abc@xyz opt]# ls -l
-rwxr-x---. 1 user1 group1 0 may 1 02:13 file1
```

OR

```
[abc@xyz opt]# chmod ugo-rwx file1        (removing rwx from all)
[abc@xyz opt]# ls -l
----------. 1 user1 group1 0 may 1 02:13 file1
```

OR

```
[abc@xyz opt]# ls -l
----------. 1 user1 group1 0 may 1 02:13 file1
[abc@xyz opt]# chmod u+rw,g+w,o+r file1        (giving read only to others, write
to group, r and w to owner)
[abc@xyz opt]# ls -l
-rw--w-r--. 1 user1 group1 0 may 1 02:13 file1
```

1.3.3.2 Octal Way

```
[abc@xyz opt]# ls -l
-rwxr-xr--. 1 user1 group1 0 may 1 02:13 file1
[abc@xyz opt]# chmod 777 file1        (giving all permissions to all)
[abc@xyz opt]# ls -l
-rwxrwxrwx. 1 user1 group1 0 may 1 02:13 file1
[abc@xyz opt]#
[abc@xyz opt]# chmod 624 file1        (giving read only to others, write to group,
r and w to owner)
```

```
[abc@xyz opt]# ls -l
-rw--w-r--. 1 user1 group1 0 may 1 02:13 file1
```

1.3.4 About Permissions

> If a user has minimal permission of "r--" for a file

- He can still open that in vi editor and write inside it (but the changes will not be saved, error will display if we try to save changes)
- He can copy file to another place

> If a user has minimal permission of "r--" for a dir

- he can only see contents of dir

> If a user has minimal permission of "-w-" (better rw-) for a file

- He can write / modify and even delete that file

> If a user has minimal permission of "-w-" (better rw-) for a dir

- he can create files/dir inside that dir

> If a user has permission of "rwx" for a file

- He can additionaly, execute the file
  [This permission is usually needed incase of script files / binary files / program files, etc.]

> If a user has permission of "rwx" for a dir

- he can go inside that dir and do all operations

> If a user has permission of "r-x" for a dir

- he can go inside a dir but cannot create any file/dir there.

We can also restrict the root (super user)

```
[abc@xyz test1]# pwd
/root/home/Desktop
[abc@xyz test1]# touch abc.txt
[abc@xyz test1]# ls -l abc
-rw-r--r--. 1 root root 0 May 12 07:50 abc.txt
[abc@xyz test1]# chattr +i abc.txt
[abc@xyz test1]# rm abc.txt
Operation not permitted!      # Although root is super user and has permissions to
write but cannot delete chattr file /neither he can change permissions of a chattr
file
```

1.3.4.1 Inheritting Permissions !

```
[abc@xyz Desktop]$ whoami
user2
[abc@xyz Desktop]$ ls -l
drwxr-xrwx. 2 root root 0 may 1 02:13 dir1      # other have r/w permission for
dir1
[abc@xyz Desktop]$ cd dir1
[abc@xyz dir1]$ ls -l
----------. 1 user2 user2 0 may 1 02:13 file1  # no permission for any one

# here although it seems, user1 can't r/w this file1 but, as user1(others) has r/w
permission for its parent dir, dir1/, so, it can inherit permissions and write and
save inside file1 using the option of over-write (:wq!)
```

> Althoug others can over-write in this case, but cannot del file1.
> Both, owner and root can do anything to file1 even though they don't have permissions. [*unless we chattr the file1*].

> **ALWAYS Remeber**

If there is file in such a dir, no matter who is owner, if anyone can over-write, the latest user that will over-write using :wq! will be shown as owner of that file.

1.3.4.2 Recrussive Permissions

Let's say we have a senerio that in a certain dir, say dir1,
we have 1000 files with permissions `-rw-r--r--`
and we want to change the permissions of all of them to `-rwxrw-r--`
We cannot do this with each file, one by one!

```
[abc@xyz dir1]# ls -l
-rw-r--r--. 1 root root 0 May 12 07:50 abc.txt
-rw-r--r--. 1 root root 0 May 12 07:50 aac.txt
-rw-r--r--. 1 root root 0 May 12 07:50 acc.txt

# we have 1000 files with same permissions (out if 1000, abc.txt is chttr file, it
is confidential)
[abc@xyz dir1]# pwd
/opt/dir1
[abc@xyz dir1]# cd ..
[abc@xyz opt]# chmod -Rv 764 dir1     # it will change permissions of all files of
folder, Recrussively (-R) and show us the changes also (-v, verbose); except the
chattr file (that will not be modified).
[abc@xyz opt]# cd dir1
[abc@xyz dir1]#
[abc@xyz dir1]# ls -l
-rw-r--r--. 1 root root 0 May 12 07:50 abc.txt  # chttr file NOT changed
-rwxrw-r--. 1 root root 0 May 12 07:50 aac.txt
-rwxrw-r--. 1 root root 0 May 12 07:50 acc.txt
```

1.3.5 About chattr

> How to check if chattr command is applied to a specific file/dir or not?

Let's suppose we have a file, say file1 in a dir, say dir1.

```
[abc@xyz dir1]$ lsattr file1        # will check attributes of file1
------------e- file1             # it shows chattr not applied
[abc@xyz dir1]$ chattr +i file1     # here i stands for immutable
[abc@xyz dir1]$ lsattr file1
----i-------e- file1             # it shows chattr file
[abc@xyz dir1]$ chattr -i file1       # to remove chattr
[abc@xyz dir1]$ lsattr file1
------------e- file1                # chattr removed
```

> No modify and No delete for chattr file/dir

`Never apply chattr for system files` , just apply on specific data file/dir that you want to secure, if applied on system file, no-one can modify it.

1.3.6 Changing Advanced Permissions_(User-Specific) | SETFACL

We have ACL (access control list); we use it to provide specific users / groups with specific set of permissions.

```
[abc@xyz dir1]# ls -l zz.txt
-rw-r--r--. 1 root root 0 May 12 07:50 zz.txt
[abc@xyz dir1]# setfacl -m u:user1:rwx zz.txt  (modify acl permissions for a
specific user for specific file)
[abc@xyz dir1]# ls -l zz.txt
-rw-r--r--+ 1 root root 0 May 12 07:50 zz.txt     # "+" shows user/group specific
acl is applied to this file
[abc@xyz dir1]# getfacl zz.txt
# file: zz.txt
# owner: root
# group: root
# user :: rw-             # it is actually of owner
# user : user1 : rwx      # acl that we applied for user1
# group :: r--
# mask :: rwx
# others :: r--
```

> Ok, now! What if we want to remove these permissions of acl if we no longer want particular user/group to use them!

```
[abc@xyz dir1]# getfacl zz.txt
# file: zz.txt
# owner: root
# group: root
```

```
# user :: rw-             # it is actually of owner
# user : user1 : rwx      # acl that we want to remove now
# group :: r--
# mask :: rwx
# others :: r--
[abc@xyz dir1]#
[abc@xyz dir1]# setfacl -x u:user1 zz.txt   # user1 removed from acl
[abc@xyz dir1]# getfacl zz.txt
# file: zz.txt
# owner: root
# group: root
# user :: rw-
# group :: r--
# mask :: r--
# others :: r--
[abc@xyz dir1]#
[abc@xyz dir1]# ls -l zz.txt
-rw-r--r--+ 1 root root 0 May 12 07:50 zz.txt   (Although no more acl here, but
why + is still there?)
```

1.3 Questions

Q.no.1 : How to provide specific user/group, certain permission for particular file/dir

We will do that using `setfacl`, see **1.3.6** for detail.

Q.no.2 Can we apply chattr to / to secure entire system?

We should never apply chttr to system file, just apply on specific data file/dir that you want to secure, if applied on system file, no-one can modify it.

1.4 Runlevels / Targets

In windows we have many modes to use;
For example:

- Start normally mode
- Safe mode
- networking mode

These types of modes in RHEL6 are called '*Run-levels*', which are **0-6**.

These are **runlevels** of RHEL6:

- runlevel 0 = HALT
- runlevel 1 = S = single user mode (safe mode) - [need no passwd to go inside]
- runlevel 2 = multiuser, without NFS (same as 3, if you do not have netwotking)
- runlevel 3 = full multiuser mode
- runlevel 4 = unused
- runlevel 5 = X11 (graphical runlevel)
- runlevel 6 = Reboot

1.4.1 Changing runlevels

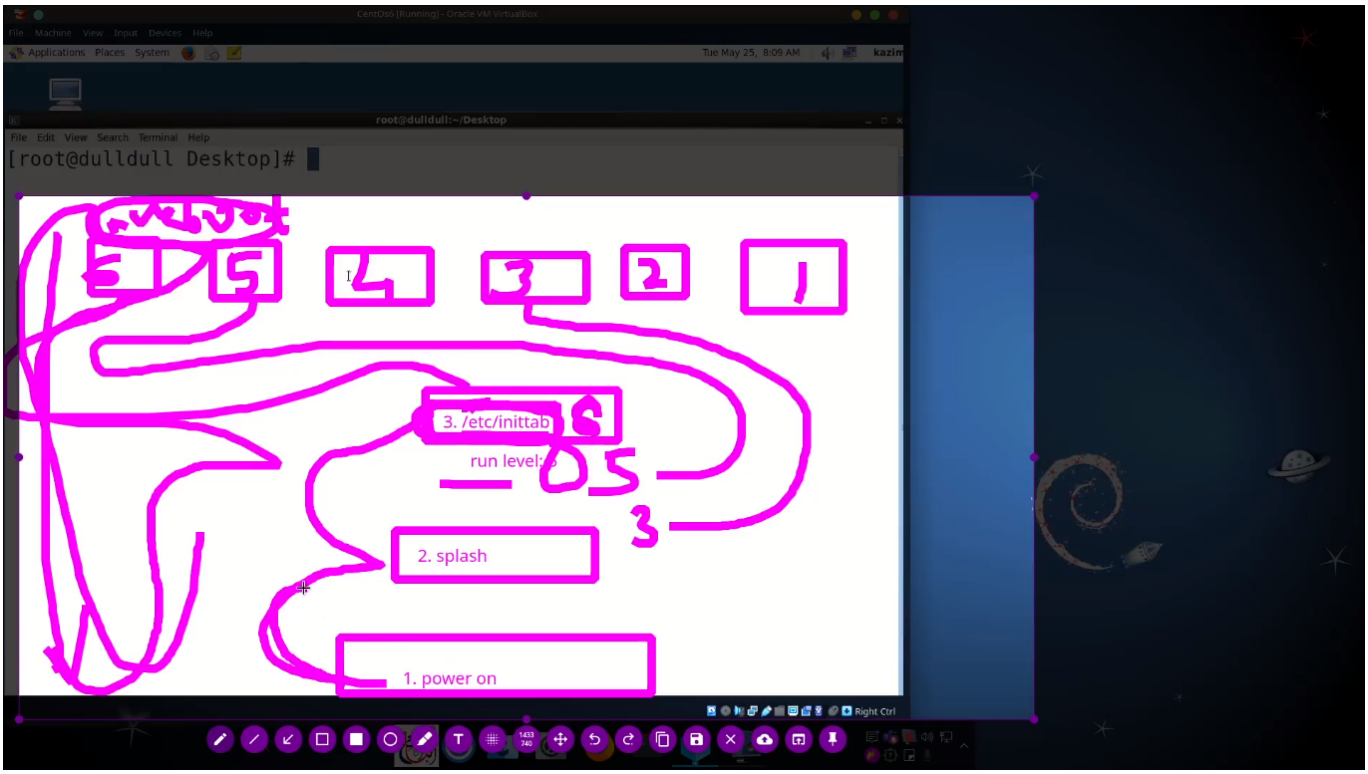
```
[abc@xyz dir1]# runlevel    # to see previous and current runlevel
n 3                         # 'n' shows you just powered on your machine (no run-
level changed recently) and '3' says, now you are at runlevel 3
[abc@xyz dir1]# init 5      # will take us to runlevel 5
[abc@xyz dir1]# runlevel
3 5         # This means we changed runlevel recently from 3 to 5
[abc@xyz dir1]# init 6      # this will reboot the system
```

1.4.2 Setting default runlevels

While we power on our machine, we see a splash screen, which than forwads and runs booting process.

Out of many, a file is run that direct us to specific runlevel by default, that is `/etc/inittab`

If we change/edit this default runlevel by modifying this file, we can land on our desired runlevel (if we set default runlevel to 6, our machine will keep on rebooting, unless we again interupt the splash screen, login temporarily by changing boot files, and making these changes permanent by editing that specific boot file for runlevels `/etc/inittab` to land on our desired runlevel again).



> I tried and successfully changed the default runlevel permenantly from `/etc/inittab`

1.4.3 Changing default runlevels (during booting)

Interrupt the splash screen while booting to change the runlevel and temporarily land somewhere else than default,

(incase of centOS v6.10)

- interrupt the splash screen using arrow key
- press `a` to modify
- look for ".....quite" in interrupted final screen and write `selinux=0 1` to interupt the booting process and temporarily enter runlevel 1;
- to enter runlevel 3, write `selinux=0 3`
- then modify default runlevel in `/etc/inittab` for making permenant changes.

1.4.4 Virtual consols

Within runlevel 2,3 and 4 we have *six consols* in each, we call them vcs (virtual consols), ranging from 1 to 6.

We can switch between them:

- Alt + F1 = switch to console 1
- Alt + F2 = switch to console 2
- Alt + F3 = switch to console 3
- Alt + F4 = switch to console 4
- Alt + F5 = switch to console 5
- Alt + F6 = switch to console 6

Whatever command we run in them, just like we are at runlevel 3 and we run a command in vcs1, then we shift to vcs2 and run another command there and again return back to vcs1, we will get our vcs1 and we will start working from exactly same place where we left (we will see our last run command there)

But, if we run a command in vcs1 and change our runlevel from 3 to 5, run some command there and return back to runlevel 3, we will not see our previously left state of vcs1, it will open fresh vcs1.

Hence, whenever we change runlevels, each runlevel will open freshed vcs and previous work over vcs of another runlevel will be wiped off.

run `ps` = To check, in which console we are (tty3 means console 3, tty4 means console 4, and so on.)

> In runlevel 5 these vcs are known as genome terminals (these are 400 in numbers means we can open upto 400 genome terminals).

> opon running `ps` , as we see /dev/tty as driver for vcs ; /dev/pts (pseudo terminals) is driver for genome terminals.

1.4.5 Targets

Unlike 7 runlevels in RHEL6, in 7_8 we have a lot many targets.
But out of them, 2 are more commonly used:

- multi-user.target (just like runlevel 3 of RHEL6)
- graphical.target (just like runlevel 5 of RHEL6)

1.4.6 Changing Targets

We can switch between targets in RHEL 7/8 with commands. i.e.,

`systemctl isolate multiuser.target`
`systemctl isolate graphical.target`

- To see, what is default target of our machine?

  run `systemctl get-default`

- To change, default target of our machine!

  run `systemctl set-default multi-user.target`
  run `systemctl set-default graphical.target`

1.4 Questions

Q no. 1 : How to reset root passwd, if you forgot it?

We will switch to runlevel 1 using `init 1` and reset root passwd using command `passwd root`.

If machine is powered off, we couldn't either get into machine (b/c it normally logins into runlevel 5 or 3 depending on particularities of OS and both ask password to login).

So while rebooting when we see splash screen, we will interupt it using arrow key and modify using ("a" in v6, and "e" in v7 and v8) and than we will write `selinux=0 1` after the word "quite" and press Enter.

It will directly take us to runlevel 1.

> There are some differences of commands for RHEL 7_8.

- "e" = to edit (in splash screen)
- "rd.break" = to switch to runlevel 1
  - right after, where if-fi script is ending, there starts a new line with ('linux16' in v7, and 'linux' in v8); we will go at the end of this long statement and write after giving a space `rd.break`
  - then DO NOT press ENTER, we press here, **ctrl+x**
- it will take us to a bash script
  - here run, `mount -o remount rw /sysroot/`
  - then run, `chroot /sysroot/`
- Now we will be in a new bash
- Now, run `passwd root` to change password
- and then run , `touch /.autorelabel` and then `exit` and then `exit`

Q no. 2 : Can we secure runlevel 1 , to stop any intruder from doing any mal-activity?

Yes, there is file `/etc/sysconfig/init` , we will edit this file.

```
                                # check last line of above file
SINGLE=/sbin/sushell            # if this, no password for runlevel 1
```

```
SINGLE=/sbin/sulogin          # if above code is comment outed, and this is last
line, single user mode
(runlevel 1) will also ask root password to login
```

Q no. 3 : What if we secured single user mode (runlevel 1) and also forgot the root password?

Yes, we can still recover using, **Secure mode**, it uses a RHEL v6 (I think v depends on which version we want to rescue) DVD or ISO file.

Q no. 4 : What are targets?

In RHEL7 and RHEL8, these runlevels are known as "*Targets*".

Q no. 5 : Can we load graphics inside runlevel 3, without switching to runlevel 5?

While being inside runlevel 3 (non -graphic multiuser mode), using command `startx`, we will allow only graphics to load, other services that are loaded while switching to runlevel 5 will not be loaded here.

Let's suppose we were in runlevel5, we went down to runlevel 3 and loaded graphic inside runlevel 3 by command `startx` and we check our runlevel againg by running `runlevel` inside genome termial of our loaded graphic interface, it will show:

```
[abc@xyz dir1]# runlevel
5 3                           # previously you were on 5, now it is runlevel 3
```

To unload this above graphics we will run command `pkill x`

But if someone has installed linux server with minimal installations, there will not have any pckg files to load graphics interface upon running `startx` , in that particular phase, graphics will not be loaded (unless we install that pckg using yum).

1.5 Partitions

First let's built some concept:

1.5.1 Hard disk

It may be any storage device.
i.e.,

- sas
- sata
- ssd
- pendrive
- SAN box (storage area network) - faster, reliable and preferred
- NAS box (network attached storage)

1.5.2 Structure of Hard disk

It is normally a circular magnetic disk/platter inside a rectangular shaped box (platters can be 2 in a box).

Inside circular disk, are many ciruclar rings, we call them 'tracks'.

Entire disk is divided by radians from centre to perimeters, the are between two consective radians that lies in between two consective tracks. forms a 'sector'.
Hard disk contains many such sectors.

Then, there is a magnetic header that writes data accross the sectors.
This written data is captured (saved) by magnetic field.

> This process, by which header writes data in 01 format is known as **Flashing**.

> The calcuations inside a hard disk is always in **cyllinderical** form.

As soon as the disk gets older, the magnetic strentgh weakens with passage of years of time, there become more chances of data missing while reading/writing.

So, warranty provided by hard disk companies also depends on quality/strength of magnets used inside disk. (i.e., 5 years, 3 years, 1 year etc.)

There are several known companies of hard disks (i.e., Hitachi, seagate, WD or western digital, etc.)

1.5.3 Device drivers

For any hardware to work properly, we need a low level programs that interacts between hardware and respective softwares, these are known as **drivers**.

These are stored inside `/dev/` , and named as;

- /dev/sda
- /dev/sdb
- /dev/sdc
- and so on....

Just like a hard drive, each partition on it also has to be associated with a driver.

Let's say, we attached a hard drive and its drivers are present with the name as, /dev/sda

If we will make 3 partitions of this hard disk, each of these three partition will be associated with drivers named as sda1, sda2 and sda3 for partition 1, 2 and 3, respectively.

Hence, a hard disk with 3 partition will have 4 drivers in total (sda, sda1, sda2, sda3)

1.5.4 Let's create partitions

The tool we use to create partitions of hard disk in hard disk is called **Dis druide**.

Commands we use for partition process:

- fdisk (most commonly used)

    - fdisk can only create, manage and del partition (cannot create partition of size > 2TB)
    - fdisk cannot mount and format partition

- cfdisk

- sfdisk parted

```
[abc@xyz /]# fdisk -l    # It will list all disk drives and its partitions with
full details.
[abc@xyz /]# ls /dev/sd*  # it will list all disk/partiton drivers
```

Task 1 :

We have to create a new hard-disk of 700MB that could be accessible by multiusers for a specific project & create a partition inside it.

1. Add disk from VM of 800MB

2. Discover this hard disk from inside our VM
   [If we add hard disk while machine ON, we have to run discover command thrice!

   ```
   echo "- - -" > /sys/class/scsi_host/host0/scan
   echo "- - -" > /sys/class/scsi_host/host1/scan
   echo "- - -" > /sys/class/scsi_host/host2/scan
   ```

   But if disk is added while machine is OFF, there is no need to run this command for discovering that disk].
   (Even after running discovery command, if we could not find new added hard-disk, just reboot the machine).

   - First `-` scans Target
   - 2nd `-` scans *scsi controller*
   - 3rd `-` scans for LUN ID
   - We run this complete discobery command thrice :
     - If we have 3 HBA cards with 2 ports each, we have to run this command six times,once for each port.

3. run `fdisk /dev/sdd` (here, sdd be the driver of newly added hard disk) to create partition of 700MB
   'm' = load help menu
   'n' = create new partition
   'w' = save changes
   'q' = quite
   'd' = delete
   'p' = print partition table

4. press `n` for 'new' and then press `p` for "primary partition"

   - (we can create maximum of 4 primary partitions in a HDD)

5. after `n` we can also press `e` if we want to create an extended partition
    - If we want to create more than 4 partitions, then we will make 1 primary partition as extended (means 3 primary and 1 extended)
    - we can make upto 1 extended in one hard disk.
    - inside extended, we can create further partitions, upto 63.
    - these prtitions inside extended are known as *logical partitions that can be 63 in maximum*
    - Infact, extended partitions are also known as *primary partitions* as these are also created over primary partition.
    - driver names between 1-4 are specified for primary partitions and an extended partition, i.e., sda1, sda2, sda3 and sda4.
    - logical partition can never be number ranged between 1-4, it will always be greater than 4 as, sda5, sda6, and so on. (logcal partition numbering starts from 5 and so on, ...)
    - If logical partitions are created inside sda4(extended), this sda4 will never be seen anywhere (i.e., `df -h`, etc.)
6. keep partition number and start cylinder as it is by default.
7. give size of partition; if we want to create a partition of 100 MBs, we will write `+100M`
8. `p` to print partition table to see if it is created perfectly or not.
9. `w` to save updated partition table (information stored in hard disk).
10. after `w` we can also run an additional command `sync` , it will take unwritten data from RAM and write it in the hard disk (better and safer to use this command also).
11. `partx -a /dev/sdd` to update kernel table about SAVED changes of partition table (then OS knows about the partition changes, this information is stored in RAM)
    - [But, if somehow, machine has rebooted just after we have saved and updated the partition table, and we could not update our kernel table, NO WORRY! because, whenever machine powers OFF, its RAM clears OFF, and when it start again, it takes informtion from hard disk and we had updated partition table stored inside hard disk, so RAM will automatically fetch that and kernel table will automatically be updated]
    - [This `partx -a /dev/sdd` command actually helps us to update the kenrnel table without rebooting our machine]
    - > We run `partprobe -s /dev/sdd` in **RHEL7_8**, at this step number 12.
12. We can use partition only after creating file system, we call it **formatting**, using command
    - `mkfs.ext4 /dev/sdd1`
      (if /dev/sdd1 is our newly created partition that needs to be formatted).
    - `mkfs.ext4 /dev/sdd2`
      (if we created another partition also)\

      [Here, formatting means creating/recreating file system (collection of meta-data, etc)]
    - We generally prefer to use `mkfs.xfs /dev/sdd2` in **RHEL7_8**

- > Generally, File System comprises of (meta-data):
  - inodes
  - inode tables
  - blocks
  - super blocks
  - mount information

13. To access it, we will add a name plate to access this partition, we say it *mount point* or *access point*.
    [we will create a special dir/ that we want to be an access point (mount point) for this partition].
    Run `mount /dev/sdd1 /tomcat/` (here, we have already created a dir tomcat/ inside root, as /tomcat/ to be added as a 'mount point').
    Now, this mount-point is connected to the partition, but this connection is temporary.
    (we can add more than 1 access point for a single/same partition)

    > We mount a partition to access its data from some point (access point / mount point)

    Because, in above example, sdd1 is access point for Kernel, we cannot access data of drive through this door, so we mount it with some access point.

    Data stored in a drive/partition can be accessed by any access point of our choice!!

```
# Let's suppose we mounted sdd1 with /tomcat/ as:
[abc@xyz /]# mount /dev/sdd1 /tomcat/
[abc@xyz /]# df -h
-----
-----
-----
sdd1 ---------------- /tomcat/
[abc@xyz /]# touch abc /tomcat/
[abc@xyz /]# ls /tomcat/
abc
[abc@xyz /]# umount /dev/sdd1    (in place of /dev/sdd1, we can write here
the address of mount-point also)
[abc@xyz /]# ls /tomcat/   # it will show nothing, b/c connection from drive
is lost
[abc@xyz /]# mkdir /opt/viyo
[abc@xyz /]# mount /dev/sdd1 /opt/viyo/
[abc@xyz /]# cd /opt/viyo/
[abc@xyz viyo]#
[abc@xyz viyo]# ls
abc            # we just connected this dir to our drive (/dev/sdc1)
               # here, abc is shown b/c this file is stored in hard disk
```

    > A bit morer understanding of above concept!

```
[root@osboxes opt]# ls
file1
[root@osboxes opt]# mkdir /opt/urwa/
```

```
[root@osboxes opt]# ls
file1 urwa
[root@osboxes opt]# cd urwa/
[root@osboxes urwa]# touch ur{1..4}
[root@osboxes urwa]# ls
ur1 ur2 ur3 ur4
[root@osboxes urwa]# cd ..
[root@osboxes opt]# mkdir /opt/zzz/
[root@osboxes opt]# ls
file1 urwa zzz
[root@osboxes opt]# mount /dev/sdd1 /opt/zzz/
[root@osboxes opt]# df -h     # to verify it is mounted perfectly
[root@osboxes opt]# cd zzz/
[root@osboxes zzz]# touch dir{1..4}
[root@osboxes zzz]# ls
dir1 dir2 dir3 dir4
[root@osboxes zzz]# cd ..
[root@osboxes opt]# mkdir nana
[root@osboxes opt]# ls
file urwa zzz nana
[root@osboxes opt]# cd nana/
[root@osboxes nana]# ls
                        # it is empty yet
[root@osboxes nana]# cd ..
[root@osboxes opt]# mount /dev/sdd1 /opt/nana
[root@osboxes opt]# cd nana/
[root@osboxes nana]# ls
dir1 dir2 dir3 dir4        # showing saved contents from drive
[root@osboxes nana]# touch pakistan
[root@osboxes nana]# ls
dir1 dir2 dir3 dir4 pakistan
[root@osboxes nana]# cd ..
[root@osboxes opt]# cd /opt/zzz/
[root@osboxes zzz]# ls
dir1 dir2 dir3 dir4 pakistan
```

> One more point to understand! (continuing from above example)

```
[root@osboxes urwa]# ls
ur1 ur2 ur3 ur4
[root@osboxes urwa]# cd ..
[root@osboxes opt]# mount /dev/sdd1 /opt/urwa/
[root@osboxes opt]# cd /opt/urwa/
[root@osboxes urwa]# ls
dir1 dir2 dir3 dir4 pakistan  # it's previous data is gone, and ;
                              # showing data from drive, where it is mounted
[root@osboxes urwa]#
[root@osboxes urwa]# cd ..
[root@osboxes opt]# umount /opt/urwa/
[root@osboxes opt]# cd /opt/urwa/
[root@osboxes urwa]# ls
```

```
ur1 ur2 ur3 ur4            # unmoutned and previous contents are restored
```

14. We will verify the mount point of new partition, using `df -h`
15. Add this driver and mountpoint into `/etc/fstab` to make this mount point, permenant (so that it may not be lost after rebooting).
    - Here, when we run `df -h` it shows results from RAM
    - but if system reboots, RAM is cleared, next time the machine will start, and we run `df -h`, it will not show us are recently created partitions.
    - we have to add this driver and mountpoint into `/etc/fstab` to make it auto-remount, whenever our machine is powered ON.

FSTAB (File System Table)

fields in `/etc/fstab` file

There are total 6 fields in `/etc/fstab` file.

- 1st field = device drivers names / UUIDs
- 2nd field = mountpoint
- 3rd field = file system type (ext2, ext3, ext4, etc)
- 4th field = permissions
- 5th field = fsck dump
- 6th field = fsck On - Off sequence : [This field determines the order in which the fsck (file system check) command checks the filesystems during the boot process.]
  The values can be:
    - 0: The filesystem will not be checked.
    - 1: The filesystem will be checked first.
    - 2: The filesystem will be checked after those with 1.

Minimum ONE SPACE is required to separate all six fields while writing.

First field of `/etc/fstab`

First field contains UUID (universal unique id) of the device driver

- `lsblk -f` is best command that lists all partitions with their names, UUIDs, FS type of partition and mountpoints.
- `blkid /dev/sdc1` , gives us uuid of the device driver
- all UUIDs are saved here, indide `/dev/disk/by-uuid/`
- `ls -l` inside `/dev/disk/by-uuid/` can give us full mapping ; to which uuid which device driver is mapped. Instead of writing uuid, we can also write directly full name of device driver
  For example : /dev/sdc1, /dev/sdc2, etc.

2nd field of `/etc/fstab`

mountpoints are well understood.

3rd field of `/etc/fstab`

FS type is also covered in questions below and is well explained within these topics (1.5 and sub topics, etc.)

4th field of `/etc/fstab`

It determines permissions for FS of specific partition.

It can be of category:

- ro | (read only)
- rw | (read / write)
- exec | (execute)
- noexec | no execute | will not allow execution of any program
- auto
- no auto | partition will not be auto remounted after reboot
- nodev | (no device file)
- nosuid | (no set user id)
- user | all users can mount/unmount partition
- nouser | only root user can mount/unmount partition
- **defaults** | it contains 7 types of permissions:
    1. auto
    2. rw
    3. dev
    4. async
    5. suid
    6. exec
    7. nouser

> If we changed permissions of a mounted partition, it will not be reflected unless we remount that partition;
> `mount -o remount /dev/sdc1`

- **Point to Remember**:
    - If there is default permission set for a partition, and we added one more (ro) to it
    - and now, 4th field become `defaults,ro` (no space in between)
    - this will give priority to `ro` permission, over `rw`, that is written inside default.
    - whatever we write after `defaults`, will be given preference; if it goes against any of the permission inside defaults
    - in this case, we (even root) cannot make changes inside this partition with `defaults,ro` permissions, unless it is changed back to `defaults`.

5th field of `/etc/fstab`

It was used by the `dump` command to determine whether a FS needs to be backed up.

- Infact, it determines, whether `fsck` will run for FS of a specific partition or not.
- In most cases, you'll see a value of 0, indicating that the FS should not be backed up.

Its value can be interpretted as:

- 0 = `fsck` will not run for this FS
- 1 = `fsck` will run over FS of this partition

6th field of `/etc/fstab`

It shows numbering :

- 0 = no auto running of `e2fsck` command
- 1 = auto running of `e2fsck` command at first place
- 2 = auto running of `e2fsck` command at second number

> Jis jis partition ki 6th field me 1 hoga un sb pr ek sath `e2fsck` command run hogi
> Aur jin jin partition ki 6th field me 2 hoga un sb pr ek sath `e2fsck` command run hogi ; '1' walon ky baad.

Adding entery in `/etc/fstab` file

```
[abc@xyz /]# vi /etc/fstab
-----
...
--------     -------     ------      ------      --    --
-------      -------     --------    ------      --    --
/dev/sdc1    /oracle     ext4     defaults    0     2     # 'yy' to copy
/dev/sdd1    /opt/viyo     ext4     defaults    0     2   # 'p' to paste & then
edit to our choice
--------     -------     ------      ------      --    --
-------      -------     --------    ------      --    --
```

Now, if we reboot our machine, this driver `/dev/sdd1` will automaically be remounted on RAM.
And `df -h` will list `/dev/sdd1` also.

Point to Remember:

- If we created a partition and its mountpoint but could NOT add that to `/etc/fstab`
- after rebooting, we again have to mount our device drivers by `mount /dev/sdc1 /opt/viyo/`

About Editing : `/etc/fstab`

- When we have made changes in `/etc/fstab`
- always test them before rebooting machine
- because if some error is there to system files, on rebooting, system can be crashed.
- so safe to take snapshot first
- and then after changing `/etc/fstab` , unmount that partition using command line (that you have added to `/etc/fstab`)

- now run `mount -a` , it will check `/etc/fstab` from hard disk
- and mount all those mountpoints that are written inside `/etc/fstab` but are not mounted at that spot!
- if it says any error,NEVER REBOOT, otherwise machine will be crahsed and enter in maintenance mode on rebooting
- upon running `df -h`, our partition with its mountpoint will be shown.

Ctrl D | maintenance mode

On rebooting we will enter Ctrl D | maintenance mode, if;

- we add a wrong enter into `/etc/fstab`
- 5th and 6th field in `/etc/fstab` should be 1 and 2, respectively.
- machine will never enter maintenance mode if we have assigned "0 0" to 5th and 6th field of our wrong entery (entery having error)
- It is because, incase of "0 0", /etc/fstab file will not bother to run `e2fsck -yc` command on that specific entery (having error).

Unmounting the disk drive

If we want to unmout the mountpoint, we have three ways

1. using direct command

- `umount /mountpoint`

2. Resolving Error | 'Device is busy'

```
[root@osboxes /]# umount /dev/sdd1        # device driver
Errot: Device is busy                     # it shows error, sometimes
[root@osboxes /]# umount /opt/viyoo       # mountpoint
Errot: Device is busy                     # it shows error, sometimes

# this means, there are some processes running in the machine in which this
partition is in use right now, so it cannot be unmounted, unless we stop all those
process

[root@osboxes /]# umount -l /opt/viyoo    # or --lazy
                                          # this will forcufully unmout the drive
```

> But this above way of forcefully unmout can cause corruption of data and uncertain behaviour of files, if there some very busy files at that time.

3. Best Resolved | 'Device is busy'

```
[root@osboxes /]# umount /opt/viyoo        # viyoo/ is mountpoint
Errot: Device is busy                     # it shows error, sometimes
[root@osboxes /]#
[root@osboxes /]# lsof +D /opt/viyoo     # "list of files" , that are being used
```

```
in particular dir
command PID USER FD ---- ---- ---- ----
---   --   --  --  --   --   --   --
---   --   --  --  --   --   --   --
---   --   --  --  --   --   --   --
[root@osboxes /]# kill PID        # PID of the proccess running over disk
[root@osboxes /]#
[root@osboxes /]# umount /opt/viyoo     # now it will unmount safely
```

1.5.5 Ways to add a new drive/partition

We have 3 options to create a new partition:

1. Purchase a new HDD from market and attach with server/machine, and later on either reboot it or discover it using discovery command.
2. Attach a USB / PD (pendrive) and server will automatically pick it up without rebooting [this is "hot plug device"]
3. we buy a SAN (storage are network : collection of HDD) and provide a LUN (logical unit number) to server.

> LUN (logical unit number) :
>
> - It is actually a disk
> - Within SAN we make a single drive by connecting all HDDs (let's say there were 10 of 500 GBs) in total.
> - Let's say making upto 5TB, then;
> - we divide this entire space into bigger portions, of 1 TB each.
> - We call them as LUN1, LUN2, LUN3, etc.

Procedure : (Via LUN from SAN)

- Host server has usually, attached cards:
  - NW (network card) attached with CAT 5/6 cable
  - HBA card (host based adapter) attached via FC cable (fibre channel cable)
- SAN also has an HBA card, HBA cards of SAN and our local server are connected via FC cable.
- After connection is established, we will perform LUN scanning (disk scanning for discovering it, which is provided by SAN)
- Let's suppose, if there were 3 luns in total, and we had only one disk in our server as sda; then we will see three more disks after this discovery step as sdb, sdc and sdd.
- When we run discovery command from *initiatior*, it goes via FC cable and scans *target*(SAN box) and then discovers *scsi controller* and *LUN ID*.
- Discovery commands performs above three poerations at once and shows us our new disk/drive in our machine as sdb, sdc and sdd if it finds three LUN_IDs from target.

SAN (storage area network)

SAN generally comprises of many hard disks, placed together.

- SAN storage is also known as *target*
- All hard disks inside SAN box are managed by *scsi controller*
- When we create LUNs of various sizez, they are provided with specific IDs; we call them LUN IDs.

SERVER/(s) that is/(are) connected to SAN is/are known as *initiator*.

1.5.6 Concept of cylinders

Data is stored in cylinders, inside a HDD.
Concept of cylinders is easier to understand in RHEL6 as compared to RHEL7 /8.

1 cylinder =~ 8 MB (if HDD size is in GBs)
1 cylinder =~ approx. 1 MB (if HDD size is in MBs)

Example 1

Let's suppose, if we attached a hard drive (let's say 'sdd') of 16 GB, we want to create partitions as, sdd1 and sdd2 of 24 MB and 96 MB, respectively.

- sdd1 will use 3 cylinders (24/8)
- sdd2 will utilize 12 cylinders (96/8)

Example 2

Just like above concept, we can calculate size of any already created partition by looking range of cylinders, which is occupied by that partition.

After running `fdisk -l`

> range of cylinder = end - start
> size of partition = range of cylinders * 8 (if HDD size in GBs)

Example 3

We can also calculate remaining free space of our 16 GB HDD.

- run `fdisk -l`
- look for highest value in "end" column
- free cylinders = total cylinder of HDD - highest end value
- free space = free cylinders * 8 (multipied with 8 b/c HDD is in GBs)

1.5.7 **fsck / e2fsck** (File System Check and Repair)

Just like 'chkdsk' in windows, we have 'fsck' in linux.

Whenever we say, 'data is currupted', actually "File System corrupts".
It happens so, because of some bad blocks / bad sectors, that are formed inside file system, with the passage of time (or some other reasons).

1.5.7.1 Checking bad blocks and cleaning disk

To check, if a pecific partition has bad block or not,
we run `badblocks -v /dev/sdc1`

- if 0 bad blocks | partition is in good state.
- if some bad blocks are found, we will clean them

we can run any of these two commands :

- `fsck /dev/sdc1`
- `e2fsck -yc /dev/sdc1` (better performance)

> These command would not work for a mouted partition (partions mounted on RAM)
> If we want to clean a partition, we have to unmount it, first.
> If unmounted, `df -h` will not show that partition, because it fetches data from RAM (not from hard disk)

1.5.7.2 Reasons | why bad blocks are created

There could be multiple reasons that why such bad sectors / bad blocks are created that makes hard disk inconsistant.

1. Unclean shutdown
    - when we suddennly hard shutdown our machine (unpluging power source / removing battery, long press power on button, etc)
    - first of all, our hard disk is affected
    - coz hard and RAM cannot sync properly and this creates bad sectors on hard disk
2. Power fluctuation
    - just like above, same problem of improper syncing of RAM and hard.
3. Wrong design of application

1.5.7.3 6th field of `/etc/fstab`

It shows numbering :

- 0 = no auto running of `e2fsck` command
- 1 = auto running of `e2fsck` command at first place
- 2 = auto running of `e2fsck` command at second number

> Jis jis partition ki 6th field me 1 hoga un sb pr ek sath `e2fsck` command run hogi
> Aur jin jin partition ki 6th field me 2 hoga un sb pr ek sath `e2fsck` command run hogi ; '1' walon ky baad.

1.5.7.3 First field of `/etc/fstab`

First field contains UUID (universal unique id) of the device driver

- `lsblk -f` is best command that lists all partitions with their names, UUIDs, FS type of partition and mountpoints.

- `blkid /dev/sdc1` , gives us uuid of the device driver
- all UUIDs are saved here, indide `/dev/disk/by-uuid/`
- `ls -l` inside `/dev/disk/by-uuid/` can give us full mapping ; to which uuid which device driver is mapped. Instead of writing uuid, we can also write directly full name of device driver
  For example : /dev/sdc1, /dev/sdc2, etc.

1.5.8 Deleting a partition

Never delete partition from in between, always delete from bottom to top.

When we delete a partition (from in between partitions), a partition is shifted ???

1.5 Questions

Q.no.1 : Why does OS shows less actual capacity of HDD as compared to labelled one?

It is because of many reasons:

1. At manufacturing end (base 10 system is used) & 1 GB = 10^9 bytes
   (1GB = 1000 MBs) & (1 MB = 1000 bytes)

   Whereas, OS uses binary values (base 2 system) & here 1GB = 1.07*10^9 bytes
   (1GB = 1024 MBs) & (1 MB = 1024 bytes)

2. File system of HDD / partitions also take space.

Q.no.2 : Can data be LOST PERMENANTLY?

- Deleting data NEVER deletes it.
- It can only be lost if over-written (previous text is lost!)

`rm -rf` in linux
`shift + del` and `format` options in windows

All such deleting options only remove the access points / reference points / (indexing) of that data.
So, mapping is lost and we cannot access it; original contents of data remain there, as such.

If somehow, we manage to restore that reference points with some tools, we can again access that data (untill unless that is not over-written).

Q.no.3 : We attached a HDD and created 6 partitions inside it (1); how many total drivers?; (2) how many file systems?; (3) how many partition tables?

Let's suppose /dev/sdb be the driver of HDD that we added.

1. There will be total 7 drivers:

   - sdb
   - sdb1
   - sdb2
   - sdb3

    - sdb4
    - sdb5
    - sdb6

2. There will be 6 file systems (One file system for one partition).

3. Only ONE partition table for ONE hard disk, which contains list of all partitions.

Q.no.4 : What is File System? | Difference between various formatting option of `mkfs`?

File System

It actually organizes / controls and manage the data.

Data of the data (metadata) majorly includes :

- inodes
- inode tables
- blocks
- super blocks
- mount information

Differences between major fils systems

When we hit TAB after typing `mkfs` , it gives multiple options, i.e.,

- mkfs.ext2
    - Came with RHEL v 4
    - Ext2 is an older file system
    - It is non-journaling type of file system
    - So, it does not support features like journaling, which helps in faster file system recovery in case of unexpected system crashes.
- mkfs.ext3
    - Came with RHEL v 5
    - improved version of ext2
    - includes journaling support
    - So, helps in faster file system recovery after a crash, as it keeps track of changes before they are committed to the file system
    - we cannot DISABLE journaling in ext3
    - we can make almost 32k dir inside partition with this FS
    - can make a single file of upto 2TB.
- mkfs.ext4, etc
    - intriduced around 2008 and came with RHEL 6
    - we can DISABLE journaling in ext4 , if needed.
    - we can make almost 64k dir inside partition with this type of FS
    - can make a single file of upto 16TB.

    - data performance is average
    - we cannot take snapshots

- mkfs.xfs

    - Came with RHEL 7/8
    - It was introduced in early 2000s but came with RHEL7 (in 2014) and RHEL8 (in 2019).
    - also supports journaling
    - data performance is very good
    - we can take snapshots

If we want to use mkfs.xfs (or else) and we are in RHEL6 (or else), we can still use it.
We just have to install that pckg (yum install....)

> Journaling :

It is actually maintaining of additional metadata records

****Q.no.5 : How to check a partition is created which type of file system?**

We can use these commands:

- `df -Th`
- `blkid /dev/sdc1` (to check /dev/sdc1)

Q.no.6 : Why to add command in `/etc/fstab` file only?

Because it is a boot process file that contains mounting details.

- It is File System Table
- `fstab` is said to be *9th critical file in linux*

Q.no.7 : How many critical files are there in linux?

Solve from chat.openai

Q.no.8 : How many fields in `/etc/fstab` file ?

There are total 6 fields in `/etc/fstab` file.

- minimum ONE SPACE is required to separate all six fields.
- 1st field starts with UUID (Universal Unique ID) / we can also write device drivers here

> 1st field = device drivers
> 2nd field = mountpoint
> 3rd field = file system type (ext2, ext3, ext4, etc)
> 4th field = permissions
> 5th field = fsck dump
> 6th field = fsck On - Off sequence

Q.no.9 : How to run `e2fsck -yc` over '/'? Can we unmount it?

No, we cannot unmount '/' in normal mode.

YES, we can unmount '/', but in Rescue mode (iso / dvd) and then run this `e2fsck -yc` command.

Q.no.10 : How many types of servers?

They are of different types:

- prod = production server | live ip ; can be accessed from outside
- dev = devloper server | used by developers
- UAT = User Acceptance Testing Server | for testing purposes
- POC = Proof of Concept | for demo

1.6 Archive / Compression Tools

1.6.1 Compression Tools

Tools we use to shrink file size / deflate files and later on expand them also.

Commands that we use are:

- `gzip` and `gunzip` (a bit fast but lower compression)
- `bzip2` and `bunzip2` (a bit fast but higher compresion - good for large files)
  (2 at the end of `bzip2` means it is 2nd version of bzip)
- `xz` and `unxz`

To what extent, a tool compresses a file depends on type of file (docs, pptx, pdf, md, ipynb, png, etc.)

> Points to remember!

- gzip indexing takes 26 nytes space
- bzip2 indexing takes 14 bytes space

So, if we compress a file with 0 size(or size of about indexing size of compression), its size will increase as indexing of compression tool will occupy some space.

```
[abc@xyz devops]# touch file{1..10}
[abc@xyz devops]#
[abc@xyz devops]# ls
file1 file2 file3 file4 file5 file6 file7 file8 file9 file10
[abc@xyz devops]# ls -l
-rw-r--r--. 1 root root 0 Jan 3 9:30 file1
-rw-r--r--. 1 root root 0 Jan 3 9:30 file2
-rw-r--r--. 1 root root 0 Jan 3 9:30 file3
....
....
-rw-r--r--. 1 root root 0 Jan 3 9:30 file10
# above files have 0 size, we have to increase them to practice compression
```

1.6.2 Redirection (>) (a relavant topic)

We can direct(send) something to somewhere we want to, by using this redirection sign >

Example 1:

```
[abc@xyz devops]# ls       # suppose we have 3 files here
file1 file2 file3
[abc@xyz devops]#
[abc@xyz devops]# ls > new   # result is directed to new
[abc@xyz devops]# ls
file1 file2 file3 new
[abc@xyz devops]# cat new
file1 file2 file3
```

Example 2:

```
[abc@xyz devops]# ls
file1 file2 file3 new
[abc@xyz devops]# date
Mon May 30 09:51:39 IST 2024
[abc@xyz devops]# date > new2
[abc@xyz devops]# ls
file1 file2 file3 new new2
[abc@xyz devops]# cat new2
Mon May 30 09:51:39 IST 2024
```

Example 3:

```
[abc@xyz devops]#
[abc@xyz devops]# history > history.txt
[abc@xyz devops]# ls
file1 file2 file3 new new2 history.txt
[abc@xyz devops]# cat history.txt
1 ls
2 ls > new
3 ls
4 cat new
5 ls
6 date
7 date > new2
8 ls
9 cat new2
```

Example 4:

```
[abc@xyz devops]# cat history.txt > ff.txt   # ff.txt is over-written with the
conetents of history.txt
```

```
[abc@xyz devops]# cat ff.txt
1 ls
2 ls > new
3 ls
4 cat new
5 ls
6 date
7 date > new2
8 ls
9 cat new2
```

> Point to remember about over-writing!

If we run bare `> ff.txt` , and do not give any file to over-write contents of ff.txt.
It will empty the ff.txt file, removes everything from ff.txt!

1.6.3 Compression

Commands that we use are:

- `zip <filename.zip> <source file>` (archive plus compression : 3rd party tool for linux)
  - ```unzip <filename.zip>``
  - `unzip -d <destination path> <filename.zip>`
  - `unzip -l <filename.zip>` just look into zip file
- `tar -cvf <filename.tar> <source file>` (archieve only : native tool of linux)
  - `tar -xvf <filename.tar>` to untar
  - `tar -tvf <filename.tar>` to see files/dir(s) inside .tar
- `tar -zcvf <filename.tar.gz> <source file>` (tar archive plus gzip compression)
  - `tar -zxvf <filename.tar.gz>`
  - `tar -ztvf <filename.tar.gz>`
- `tar -jcvf <filename.tar.bz2> <source file>` (tar archive plus bzip2 compression)
  - `tar -jxvf <filename.tar.bz2>`
  - `tar -jtvf <filename.tar.bz2>`
- `tar -Jcvf <filename.tar.xz> <source file>` (tar archive plus xz compression)
  - `tar -Jxvf <filename.tar.xz>`
  - `tar -Jtvf <filename.tar.xz>`
- `file <any filename>`
- `zcat <any zip file>` ro read any zip file (.zip, .gz, .bz2, etc)

TAR (Tap Archieve)

Now, let's increase our file sizes first.

```
[abc@xyz devops]# ls -l       # let's suppose we have 3 files here!
-rw-r--r--. 1 root root 0 Jan 3 9:30 file1
-rw-r--r--. 1 root root 0 Jan 3 9:30 file2
-rw-r--r--. 1 root root 0 Jan 3 9:30 file3
[abc@xyz devops]#
[abc@xyz devops]# man ls > file1
```

```
[abc@xyz devops]# man ls > file2
[abc@xyz devops]# man ls > file3
[abc@xyz devops]# ls -l
-rw-r--r--. 1 root root 10088 Jan 3 9:50 file1
-rw-r--r--. 1 root root 10088 Jan 3 9:50 file2
-rw-r--r--. 1 root root 10088 Jan 3 9:50 file3
```

commands	Fx
`gzip file1`	it will compress file1 into file1.gz
`gzip *`	compress all files inside pwd into .gz files
`gunzip file1.gz`	it will unzip file1.gz to file1
`bzip2 file1`	it will compress file1 into file1.bz2
`bunzip2 file1.bz2`	it will unzip file1.bz2 to file1
`xz file1`	it will compress file1 into file1.xz
`unxz file1.xz`	it will unzip file1.xz to file1
`zip dabba.zip abc.txt ff.txt`	It will archieve abc.txt and ff.txt into dabba.zip
`zip fun.zip *`	It will archieve (gather) all files/dir(s) of current pwd into fun.zip
`zip fun.piz *`	we wrote .piz, it will also work 100% coz Ext. DO NOT matters here, it simple pose a bit difficulty for users to distinguish such zipped files (So, we can write any Ext. i.e., .plusk, .stuu, .viyoo, etc)
`file fun.piz`	to check which type of format is of this file (this time it will say fun.piz is a zip archive file)
`unzip fun.zip`	it will unzip fun.zip in pwd
`unzip -d /opt/test3/ fun.zip`	fun.zip will be unziped inside /opt/test3/
`zcat file1.gz`	to read the compressed file, without inflating it
`unzip -l fun.zip`	it will list all those files that are present inside archive

commands	Fx
`tar -cvf abc.tar file1 file2`	archive file1 and file2 into abc.tar
`tar -cvf fun.tar /opt/test/*`	archive everything inside /opt/test/* into fun.tar
`tar -cvf magic.tar *`	archive everything presnet inside pwd into magic.tar
`tar -tvf fun.tar`	will only show us list of files that are archived inside this fun.tar
`c*`	it will select all all files of pwd that start with c
`*`	it will select entire content of pwd
`/opt/test/*`	will select everything inside /opt/test/
`/opt/test/ *`	we are giving two sources here, it will select full dir of /opt/test/ and everything inside current pwd COZ their is space before * in this command

1.6 Questions

Q.no.1 : How to add one more file to already created .zip file?

Q.no.2 : How to extract specific file/(s) from .zip file?

Q.no.3 : How to change compression ratio for .zip file?

Q.no.4 : How to password protect .zip file?

Q.no.5 : Interview questions moves about what?

Mostly, interviews spin around following topics:

- apache
- boot process
- critical files of linux
- backup tools (cp, Rsync, cpio, etc.)
- lvm
- mysql
- suid
- stickybit

1.7 Backup Tools

Normally we take backups in remote device, it can be;

- another remote machine
- SAN (storage area network)
- NAS
- Tape drive / LTO (linear tape operator)

We can take backup with lot many tools, i.e.,

- Rsync (very powerful tool) (used for remote backup) (it uses port 22 ssh protocol)
- cp (cp -rv) [simple cp can only take backup of files (not directories)]
- tar -cvf
- zip

`rsync` is much better than cp;

- because if we use cp to take backups, everytime we take backup and run cp command, it will over-write all files at destination path and asks for each and every file.
- Moreover we bare `cp` do not copy dir to destination path, we have to add a flag as `cp -rv`
- although rsync also takes flag to work with directories also, as `rsync -pravz` (here -p = preserve permissions; owner remains same as in source location)

1.7 Questions

Q.no.1 : What to do if you are assigned to take backup everyday at sharp 02:00 am ?

Backup is actually stored in another machine (remote machine).

- We will design a script using cron scheduler.
- And put inside it, entery of Rsync protocol.
- It will run automatically sharp at a given time.
- Rsync actually works via SSH protocol using port 22.

Q.no.2 : What are types of backup? Rsync does which type of backup?

Backup can be of different types:

1. Full backup
    - A full backup involves copying all selected files and data in a system to another storage location.
    - It creates a complete and independent copy of the entire dataset.
    - Fast restores since all data is in one place, simplicity.
    - Consumes more storage space and time, especially for large datasets.
2. Incremental backup
    - Only the data that has changed since the last backup (full or incremental) is copied.
    - Efficient in terms of storage space as it backs up only changes.
    - Faster backup, less storage space usage.
    - Restore process can be slower since you need the last full backup and all subsequent incremental backups.
3. Differential backup
    - Backs up all changes made since the last full backup. Unlike incremental, it doesn't rely on previous differential backups.

    - Requires less storage space compared to a full backup but more than incremental.
    - Faster restores than incremental as you only need the last full backup and the last differential backup.
    - Over time, differential backups can become larger.
4. Cumulative backup
    - Similar to incremental, but it includes all changes made since the last full backup.
    - Each cumulative backup includes all changes since the last full backup, not just changes since the last backup.
    - Faster restores than incremental, but slower than differential. Each cumulative backup is a standalone backup.
    - Over time, cumulative backups can become large, and the restore process may take longer.

> `Rsync` performs incremental type of backup.

Q.no.3 : Does `Rsync` also compresses the file while doing backups? If NOT what if we want to?

No (according to Al-nafi tutor)

`Rsync` does not compresses the files while doing backups.

But if we want compression also, alongside backups, we have to use multiple commands using pipe. i.e.,

- `Rsync | zip`
- `Rsync | tar -zcvf` (for gzip compression)
- `Rsync | tar -jcvf` (for gzip compression)
- `Rsync | tar -Jcvf` (for xz compression)

Yes (according to chatgpt)

`Rsync` can compress the files while doing backups.

- `rsync -pravz /source/directory/ /destination/directory/`

Here:
-a = archive
-v = verbose
-z = compression
-p = preserve permissions : owner remains same as in source location -r = takes recrusive backup ; includes all dir/sub-dir with their files

Q.no.4 : Write down 10 examples of `rsync` ?

Q.no.5 : How `rsync` works?

- ``rsync``` compares source and destination files modify time
- If any change is observed in source, it will copy only incremental data from source to destination
- If modify time of any file/dir is changed at source, it will copy only that changed file (that will be showing changed modify time than that of present at destinaation)

- In this case, inode number of such file that is over-written at destination, is changed than of that, present at source.
- normally it has nothing to do if something extra is present at destination
- although we can add a flag to properly synchronize source and destination
- `rsync -pravz --delete` will delete any extra file present at destination, synchronizing both, source and destination

NOT WORKING this `--delete` YET :

```
[root@osboxes /]# rsync -pravz --delete /opt/test1/* /usr/backup1
```

Q.no.6 : How does `rsync` works for remote backup ?

We will run this commands:

```
[root@osboxes /]# rsync -pravz --delete /opt/test1/* 192.168.87.1:/usr/backup1
```

The above command is as like this :
`rsync -pravz --delete /opt/test1/* <remote_ip:destination_address>`

If we want to store backup in a remote machine but inside that, we want backup to be stored for specific user. We can do that too by modifying our command.

Let's suppose we want to store backup in user_new of remote machine.

```
[root@osboxes /]# rsync -pravz --delete /opt/test1/*
user_new@192.168.87.1:/usr/backup1
```

`rsync` is very safe to use, because if we run this command from root, it will ask username/password of remote machine/server _ **credentials of 65 server of destination** while transfering backups using *ssh protocol via port 22*.

Q.no.6 : Name some of the important commercial(paid) backup applications used by Enterprises and some open source tools/applications that we can use in linux

Commercial(paid) backup applications

Companies usually pay for these tools and they include :

- veritas netbackup
- windows NT backup
- TSM (Tivoli Storage Manager)
- VEEAM
- DELL Networker

- EMC Networker

Open source tools/applications

- AMANDA
- ZAMANDA
- BACULA
- BackupC

1.8 Piping

We use pipe "|" in linux to pass the output of first command as an input of second command.

1.8.1 Example1 (with `wc`)

```
[root@osboxes etc]# ls
---- --- --- --- ---- --- --- --- --- --- (it will list all files/dir present
inside it)
[root@osboxes etc]# wc -l     # word count (-l for new line count)
                              # it has no input ; so will not work as bare command
[root@osboxes etc]# ls | wc -l
274
```

1.8.2 Example2 (with `less`/`more`)

```
[root@osboxes etc]# ls /etc/*
---- --- --- --- ---- --- --- --- --- --- (it will list everything inside it :
will not fit in page/screen)
[root@osboxes etc]# ls /etc/* | less     # "less" will page fit the output
                                         # single line scrolling

[root@osboxes etc]# ls /etc/* | more     # "more" will page fit the output
                                         # page-page scrolling
```

1.8.3 Example3 (with `grep`) | Use of `grep`

`grep` is a powerful string-wise search command.

`grep -i` = search insenstive string
`grep -n` = also show line number of output
`grep --color` = result will be colored
`grep -A1` = will show results and one line below it
`grep -A2` = will show results and two lines below it
`grep -a1` = will show results and one line above and below it
`grep -a2` = will show results and two lines above and below it

```
[root@osboxes etc]# grep user1 /etc/passwd   # it will search /etc/passwd for
'user1'
user1:x:500:500.........
[root@osboxes etc]# history | grep -i --color urwa   # will search 'urwa' in
history and color it
```

We can also use mutiple pipes, within a single command:

```
[root@osboxes etc]# history | grep -i urwa | wc -l    # number of lines with
'urwa', in history
```

**1.9 Some important tools (`find` / `locate` / `sed`)

1.9.1 `find` usage

`find` is very powerful command in linux.
It has multiple usage, some of them are listed below:

1.9.1.1 Search for Files/dir by Name

```
[root@osboxes etc]# find /path/to/search -iname abc.txt
[root@osboxes etc]# find . -iname abc.txt                   # "." mean pwd
[root@osboxes etc]# find /path/to/search -iname dir1
dir1
[root@osboxes etc]# find /path/to/search -type f -iname abc.txt
# will only find files with this name


[root@osboxes etc]# find /path/to/search -type d -iname dir1
# will only find dirs with this name
```

1.9.1.2 Search for Empty Files and deleting them

Searching for Empty Files:

```
[root@osboxes etc]# find /path/to/search -type f -empty
```

Deleting Empty files, simultaneously:

```
[root@osboxes etc]# find /path/to/search -type f -empty -delete
```

1.9.1.3 Searching Files with a Specific Extensions and deleting them [NOT_WORKING]

To search files with specific extensions:

```
[root@osboxes etc]# find /path/to/search -type f -name "*.log"
```

To search & delete files with specific extensions:

```
[root@osboxes etc]# find /path/to/search -type f -name "*.tmp" -delete
```

1.9.1.4 Search for Large Files (e.g., >100MB)

```
[root@osboxes etc]# find /path/to/search -type f -size +100M
```

1.9.1.5 Search for Files m/a/c time in the Last N Days / N minutes

For Days:

```
[root@osboxes opt]# find /opt -type f -mtime -7
[root@osboxes opt]# find /opt -type f -atime -7
[root@osboxes opt]# find /opt -type f -ctime -7
# will find all those files inside /opt that were modified/accessed/changed in
last 7 days
[root@osboxes opt]# find /opt -type f -mtime +7
[root@osboxes opt]# find /opt -type f -atime +7
[root@osboxes opt]# find /opt -type f -ctime +7

# will find all those files inside /opt that were modified/accessed/changed before
last 7 days (7 days before)
```

For Minutes:

```
[root@osboxes opt]# find /opt -type f -mmin -60
[root@osboxes opt]# find /opt -type f -amin -60
[root@osboxes opt]# find /opt -type f -cmin -60

# will find all those files inside /opt that were modified/accessed/changed in
last 60 minutes
[root@osboxes opt]# find /opt -type f -mmin +60
[root@osboxes opt]# find /opt -type f -amin +60
[root@osboxes opt]# find /opt -type f -cmin +60

# will find all those files inside /opt that were modified/accessed/changed before
last 60 min (1 hr before)
```

1.9.1.6 Search for Files Based on Permissions

```
[root@osboxes opt]# find /opt -perm 777     # will find all files with "full
permissions for all" in /opt

[root@osboxes opt]# find /opt -perm 777 -exec chmod 666 {} \;
# this will find all certain files and change their permissions to 666
```

1.9.1.7 Search for Files Based on Inode Number

```
[root@osboxes etc]# ls -il       # long list with inode numbers
[root@osboxes etc]# find /path/to/search -inum 24914
```

1.9.1.8 find tasks of a specific user in entire machine

```
[root@osboxes etc]# find / -user osboxes      # list all tasks of "osboxes" in "/"
/home/osboxes/...........
/home/osboxes/...........
/home/osboxes/...........
/home/osboxes/...........
/home/osboxes/...........
# and so on.....till last process of "osboxes"
```

1.9.1.9 find tasks of a specific user in specific path only

```
[root@osboxes opt] find /opt -user osboxes   # list files of "osboxes" inside
'/opt'
[root@osboxes opt] find . -user osboxes     # same as above ("." means pwd)
[root@osboxes opt] find -user osboxes       # same as above (by default run in
pwd)
```

1.9.1.10 `find` in combinition with other commands

```
[root@osboxes dir1]# find . -user osboxes
abc.txt

# now let's execute an operation over this output of `find`
[root@osboxes dir1]# find . -user osboxes -exec rm -i {} \;
```

Splitting this recent command:

- > `find . -user osboxes -exec rm -i {} \;` OR
- > `find . -user osboxes -exec rm -rf {} \;`
- `find` = first command
- `.` = path for first command
- `-user` = for specifying a user
- `osboxes` = user
- `exec` = acts as a pipe here, to execute 2nd command on Output of `find`
- `rm` = second command
- `-i` = flag for interaction : asks before removing each file
- `-rf` = will remove forecfully (will not as, even if used along `-i`)
- `{}` = It's a placeholder & gets replaced by each output of `find`
- `\;` = helps OS understand, `-exec` command is terminated here

1.9.1.11 `find` in combinition with other commands

```
[root@osboxes dir3]# ls

[root@osboxes dir3]# find /opt/dir2/ -perm 777 -exec mv {} /opt/dir3/ \;

[root@osboxes dir3]# ls -l
-rwxrwxrwx. 1 root root 123k Jan 18 11:43 abc.txt

# We can perform a range of such processes with `find`
```

1.9.2 `locate` usage

- `locate` is much fast as compared to `find`
- but in some particular cases, it cannot give real time output!
- becaue `locate` take data directly from database
- `locate` maintains its own database
- it database file is in a flat file stored : `/var/lib/mlocate/mlocate.db`
- and database is updated at the time of reboot (automatically)
- so, in a running machine, if we create a new file and then try to find it using `locate` command, it cannot find
- unless we reboot our machine (to update database)
- or less we have to run `updatedb` command to update database
- after `updatedb` there is no need to reboot, `locate` will work perfectly

```
[root@osboxes etc]#
[root@osboxes etc]#
```

1.9.3 `sed` usage

- It is known as Stream editor

- A command-line tool for parsing and transforming text.

```
[root@osboxes etc]# sed -i 's/oldword/newword/g' filename.txt
[root@osboxes etc]# sed -i 's/oldword/newword/' filename.txt
[root@osboxes etc]# sed -i 's/oldword/newword/gI' filename.txt
```

> sed = stream editor
> -i = in place editing | saves changes in the file
> /g = if `g` skipped | `sed` will only substitute one `oldword` in a single line (`/g` means all `oldword` of each line) /I = adding `I` makes substitution case insensetive. (so, use carefully!)

1.10 Processes and Deamons

Any thing that runs in a machine is called process and all processes run inside RAM.

There are enormous number of programs:

httpd = apache | handles web serving
MySQL = RDBMS | open-source relational database management system
docker = container service
kubernetes = provides solution for orchestrating and managing containerized applications

- open file = programs or files that are loaded in ram

To check how many open files are there:

```
[root@osboxes etc]# lsof      # list of files/processes that are on RAM
---   ---   ---   ---   ---
---   ---   ---   ---   ---
and so on...
```

> **Erandom Program**

Erandom program assigns and manages PID for each process.

- Each process/service is assigned a random PID
- When we `restart` the service or RESTART our machine, PIDs of most of the programs (almost 80%) will be changed from previous ones.
- PID of "init" is always 1.
- `init` is called *Father of all processes* uptil RHEL6
- `systemd` is the *Father of all processes* ; in RHEL7_8

> If PID of `init` is 1, its PPID is 0.
> Parent process of `init` with PID 0 is known as "System idle process" (swapper or scheduler).

1.10.1 Processes | monitering Tools

There are lot of performance monitering tools.

For Example :

Process monitering tools ;

- `ps`
- `pstree`
- `lsof`
- `du`

Some other process monitering commands ;

- `top`(processes and their loads over cpu, memory, etc)
- `htop` (better version of top, with colors)
- `iostat` (hdd load)
- `vmstat` (mem. and swap)
- `iometer`
- `ipref` (network bandwith monitering)
- `tcpdump`

1.10.1.1`ps`

- a very powerful command
- Helps us see, how many process are running in RAM.
- most common used flags:
  - `ps -el | less` = gives detailed list of process in 15 columns (`less` to screen-fit)
  - `ps -ef` = gives precise information of processes, in 8 columns
  - `ps -aux` = details of process, some unique details than `ps -el`
  - `ps -aus` = gives detail of flags used with `ps`
  - `ps -el | wc -l` = to see total number of running processes
  - `ps --help`
- these all above commands fetch details of running process from RAM

1.10.1.2 Searching PID of a program and killing a process

To search PID of any program, we can run this command;
(Let's suppose we want to search PID of firefox)

- `ps -el | grep -i firefox`
- `ps -el | grep -i fire`
- `ps -el | grep -i fire*`

OR

- `pgrep firefox`
- `pgrep fire`
- `pgrep fire*`
  (`pgrep` will show numerical PID alone, will not give us complete process name in output)

OR

- `pidof firfox` (will only shows pid of particular process if complete exact name is given)

Here in above commands ;

- `grep` = find out all process that has 'fire' in their name
- `-i` = regardless of case of letters

> **To kill a specific process**:

- First, find the pid of that process, using `ps -el | grep -i process_name`
- then run : `kill PID`

OR

- `kill -9 PID` (to kill process: forcefully)

OR

- `pkill fire` (killing process by its name) | but this is risky.
- `pkill firefox`

In last commands, if we run `pkill fir`, it will kill all process whose name starts with 'fir'.

1.10.1.3 Checking CPU & memory info.

To check information of CPU and memory, we can run these commands:

- `cat /proc/cpuinfo` (for cpu info)
- `lscpu` (this command also lists cpu info)
- `lsusb` (to list usb info)
- `cat /proc/meminfo` (for memory info)
- `cat /proc/cpuinfo`

> `/proc/` contains process files navigated from RAM:

- this dir becomes clear, when we power OFF our machine
- whatever processes are running in RAM, are present inside `/proc/`
- whatever commands we run in RAM, RAM shows us output from `/proc/`
    - `top` commands shows us information, taking data from `/proc/`

1.10.1.4 `lsof` (List of Files)

- `lsof` = gives us detailed list of running process.
- `lsof /opt` = (running process inside /opt)
- `lsof +D /opt/viyoo/` = (processes running in particular dir)
- `lsof .` = process running in 'pwd'

1.10.1.5 `pstree` (tree of process)

`pstree` gives us information of running process in hirarichal way with the help of lines.

- tree that we see do not holds, realtime process ; it rather shows us the state of machine at the time of running `pstree` command.
- `pstree` is good, to see a broader look of entire processes of machine
- `pstree | less` gives screen fit view of tree.
- it shows, if we kill `init`, all those process that are started by `init` will be killed spontaneously.

1.10.1.6 `top`

- `top` is a very powerful command that provides us real-time view of system processes.
- `top` is primarily used for monitoring system resource usage (cpu and memory usage, etc.)
- we can sort processes based on different criteria, kill processes, and perform other actions without exiting the program.
- If we want to use redirect sign with it, we will use it as `top -n1 > top.txt`
- `top` do not takes the commands such as `less` , `more` ,etc. alongside with pipe.

`top` shows summaries in horizontal way at the top and details of individual process in the below columns.

- Once you are inside interface of the "top":
  - `u` = will sort the process according to specific username we type after pressing `u`
  - `c` = shows the full command lines with paths in last column
  - `z` = will highlght all important things on the "top" screen
  - `M` = will show the maximum memory taking process at top of all
  - `h` = to open help menu
  - `x` = to arrange on basis of specific column (perhaps)
  - `Shift + <` , `Shift + >` = to toggle between columns
  - `y` = running processes
  - `z` = change color; mono-color
  - `b` = bold / highlight (will work only alongside 'x', or 'y')

1st line of `top`

```
09:02:34 up 1:14, 3 users, load average: 0.00 0.01 0.02
```

Here is split of above output;

- 09:02:34 = total runtime of machine
- 1:14 = up-time (T since last reboot)
- 3 users = no. of users (2 if 1 terminal open, 3 if 2 terminals open, and so on.)
- 0.00 = average load in last 1 minute
- 0.01 = average load in last 5 minute
- 0.02 = average load in last 15 min. (to see load of past 30 minutes, use `sar`)

2nd line of `top`

```
Tasks: 190 total, 1 running, 189 sleeping, 0 stopped, 0 zombie
```

It tells about total processes and their current(real-time) state. It may be:

- R = running
- S = sleeping
- Zombie = uninterruptable
    - a dead process waiting to be killed by parent
    - a terminated process that has not yet been completely removed from the system and is sitting inside RAM

How to kill zombie processes?

- if these are less in numbers, nothing to be worried (perhaps)
- but if count increase, "reboot" is good option to kill/remove all such process

3rd line of `top`

```
Cpu(s): 0.7%us, 0.3%sy, 0.0%ni, 99.2%id, 0.2%wa, 0.0%hi, 0.0%si, 0.0%st
```

Important value to look for is %id

- 100%id = 100% cpu is free
- 99%id = 99% cpu is free (idle)

Others include:

- us = user percentage of load
- sy = % system usage
- ni = nice value | priority value of processes
- wa = percentage of CPU time spent waiting for I/O operations to complete
- hi = hardware interupt (this value increases when issue in hardware, NIC, etc)
- si = software interupt
- st = percentage of CPU time "stolen" from a virtual machine

Nice value | (concept)

- In linux, if we do a task using `cp` command and it takes 2hrs. We can do same process in 1.5 hrs by speeding up process.
- CPU actually shifts the process from a bigger(wider) cycle to smaller one (an inner ring cycle) which completes earlier that that of outer ones.
- Likewise we can speed up and slow up processes by our choice.
- We set priority of process in CPU by changing their nice value.
    - `ni` = "-20" | highest priority for any process (its PR value =0)
      [The lowest possible PR value=0, and it corresponds to the highest scheduling priority]
    - `ni` = "+19" | lowest priority value for any process (its PR value =39)
- Changing nice value of a process:
    - get inside `top` interface

    - press `r`
    - enter pid of a process to renice its value
    - enter nice value (generally by default it is 0) | PR = 20
    - if we enter `-10` as nice value (it PR=20-10=10) | priority is set from 20th to 10th

Normally, nice value = 0, and PR=20, for all processes:

- We can run any command and assign its nice value alongside command:
    - `nice -n 10 ls`

Co-relation between Nice value and Priority value

> Nice Value | (niceness):

- The nice value range from -20 to +19.
- A lower nice value means a higher priority.
- It represents how "nice" or "friendly" the process is to other processes.
- > The range is from -20 (highest scheduling priority | PR=0) to +19 (lowest scheduling priority | PR=39).
- A higher nice value means a lower priority.
- By default, most processes start with a nice value of 0.

> Priority (PR):

- The priority (PR) is a value calculated based on the nice value.
- `PR = 20 + nice value` | So, PR can range from 0 to 39.
- A lower nice value results in a higher PR.
- The PR value is what you see in tools like top or ps when displaying process information.
- *For example*
    - if the nice value is -10, the PR would be 20 - 10 = 10.

Renicing a process | changing nice value

Let's go through the process of renicing a process using the top interface:

- Open top by running the top command in the terminal.
- Press r to enter the renice mode.
- Enter the PID of the process you want to renice.
- Enter the new nice value.
    - If you enter -10 as the nice value, the PR would be calculated as 20 + (-10) = 10.
    - If you enter +10 as the nice value, the PR would be calculated as 20 + 10 = 30.

OR, we can also change nice value / (if not already running) start a command with specific nice value using command prompt:

- `nice -n -10 ls`

So, in `top`, a process with a lower PR value has a higher priority. If you want to give a process a higher priority, you would assign it a lower nice value (e.g., -10), resulting in a lower PR value (e.g., 10).

4th line of `top`

```
Mem: 3923316k total, 1148368k used, 2775072k free, 67952k buffers
```

To see details of memory only, we can also run `free -m`

> Buffer memory:

- It refers to the portion of physical memory used for buffering I/O operations inside RAM.
- It is used to temporarily store data being transferred between the system's memory and I/O devices.

5th line of `top`

```
Swap: 8191996k total, 0k used, 8191996k free, 380908k cached
```

> Cached:

- The amount of swap space that is being used to cache data
- When the system needs to free up space in physical RAM (because it's getting full or to optimize performance)
- It may transfer less frequently used data from RAM to swap space
- This data is then marked as "cached" in the swap space.
- When the system needs that data again, it can be read back into RAM from the swap space.

Columns inside `top` interface

- PID = process ID
- USER = owner of program
- PR = priority value
    - Its value is inversely related to schudling_priority of a process in cpu
    - Its means, higher the PR value, later it will be executed by cpu.
    - and a process with lowest PR value (PR=0) will have highest scheduling priority in cpu and executed first of all.
- NI = nice value
    - well explained by now.
- VIRT = virtual memory | swap
- RES = actual resedential memory
- SHR = shared memory
- S = state of process (running, sleeping, stopped, zombie, etc)
- %CPU = cpu utilization
- %MEM = memory utilization
- TIME+ = process running time

- COMMAND = process name

1.10.1.7 du (Incomplete)

link to its examples

du is commonly used by system administrators to automate monitoring and notification programs that help prevent directories from becoming full.

Examples of du usage:

1. Type du to display usage in current directory :

```
du
```

FORK BOMB (VERY DANGEROUS COMMAND)

`:() { :|:& };:`

- This is known as "Fork Bomb"
- This will spontaneouly open immense no. of files and process that will occupy entire space of RAM and CPU
- Thus, causing our machine to crash, suddenly.

1.10.1.8 Cores of processors

Processor may be single, dual, quad, octal core etc

- single core = can manage only one application/program/process at the spot
- dual core = can manage 2 programs at the same time
- quad core = can manage 4 programs at the same time

> **Concept of load** :

If we have a dual core processor, it can manage 2 processes at the same time.
But if somehow, there comes 3rd program that needs to be run over processor, it will wait in the que, until any of the core is free.

- load = 1 : means there is 1 program in que
- load = 2 : means there are 2 programs waiting in que
- and so on...

1.10.2 SWAP | Overview

About SWAP | Is it compulsory? | Its FS ?

SWAP :

- SWAP is virtual memory

- a partition on hard disk
- a partial expension of RA

SWAP | Compulsory or not :

- No, SWAP is not Compulsory to have SWAP always:
- if we have a good healthy RAM in our machine, we can install linux without SWAP
- but in some cases, there comes some programs / application that requires SWAP space to work efficiently (even is RAM is not fully occupied)

FS in SWAP :

- SWAP uses virtual file system (virtual FS)
- 'virtual FS' is a special type of FS that helps store process in SWAP, unlike other FS that stores data in other partitions ('pp', 'e', etc.)

Concept of SWAP

It is actually a space on hard disk that helps RAM to load more application that its capacity.

For Example:

- If we have a RAM of 128MB in our machine and we opened two programs, firefox and word.
- size of each of these two is 64MB.
- when loaded together, RAM is 100% occupied.
- Logically we cannot open any third application (unless one of these two is closed)
- But, we want to open a third program (say ppt) without closing any of these two.
- So, we specify some space in hard disk (double the size of RAM!!).
- When we open ppt, word moves to that specified space on hard disk (SWAP) and becomes passive (do no closes).
- Hence techincally, **Swaping** means changing location of files between RAM and Hard disk (a file/program/application moves from RAM to Hard disk and when needed, such moved applications / programs move back to RAM).
- Active applications will always be in RAM.

Here is a picture showing above concept of SWAP

SWAP size | (why double of RAM ?)

Let's suppose, our 500MB RAM is full and we also have some applications (of almost 500MBs) in swap as passive programs:

- now if we open any new application (xls) by double clicking it (it performs two actions):
    - firstly an already running / active program from RAM moves to hard disk (swap) to create space for xls.
    - then xls is loaded over RAM
    - even if we want to load again any of those passive applications present in swap, same procedure is followed (firstly some applications move from RAM to SWAP to create space and then our desired application is loaded in RAM, from SWAP)
- So, generally, size of SWAP should be double of RAM, at minimum.

But this relation of size is not fixed, if we have a RAM of 64GB and is not overwhelmed, most often.

- We can also have SWAP of less size as that of RAM (~ 32GB) (again it all depends on size of application that we are using most often)
- it is mostly written in recommendation of application
- redhat also has a documentation for minimum recommendations for SWAP

Making SWAP memory | extending SWAP

Generally, It can be extended by three way:

1. `fdisk` partition method
2. file way
3. file way with different command

1. Making/Extending SWAP | `fdisk` partition method

We can make (or extend, already existing) swap in Linux by following steps:

- attach a disk / or virtual disk
- run `fdisk -l` to see, disk is discovered (otherwise run its discovery command)
- run `fdisk /dev/sdc` and create a normal parition `sdc1` (as we've done already)
- press `p` to print partitions (verifies, if created)
- change type of disk we created now:
    - within interface of `fstab`, press `t`
    - then write no.(serial number) of partition, we gonna change type ('1' in this case | for sdc1)
    - write there, 'Hex code :' `82` (just write `L` if want to see list)
        - it is actuall, id of a partition
            - id = 82 for swap parition
            - id = 83 for normal partition
- type `w` to write changes in partition table and come out
- run `partx -a /dev/sdc` to write these changes in kernel table
- here, we will not add FS, using a normal formatting (using `mkfs.ext3` or esle...)
- run `mkswap /dev/sdc1` = it make swap of `/dev/sdc1`
- by now, two partitions are there
    - one present already (say, `/dev/sda3`)
    - other, we created now (i.e., `/dev/sdc1`)
- We will merge both of these partitions now (by entery into `fstab`)
- run `vi /etc/fstab`
- write such an etery there in the end, `/dev/sdc1 swap swap default 0 0`
- here we do not run `mount -a` coz not a normal partition
- instead, we will disable-enable the swap, to merge them both using:
    - `swapoff -a`
    - `swapon -a`

2. Making/Extending SWAP | file way

3. Making/Extending SWAP | file way with different command

Checking SWAP memory

- `free -m` = details of usage of SWAP memory
- `swapon -s` = to see "swap" is made on which partition
- `fdisk-l /dev/sda` = shows all partitons of our main disk 'sda' | swap will also be shown

Checking SWAP usage of application

Let's first open firefox

```
[root@osboxes /]# pidof firefox
4452
[root@osboxes /]# cd /proc/      # it constain dirs of all pids
[root@osboxes proc]# cd 4452/    # enter firefox dir
[root@osboxes 4452]# ls status   # will list status file
[root@osboxes 4452]# cat status | less  # look into status file, it will show all
concerned info
```

Alternate direct command:

```
[root@osboxes 4452]# pidof firefox
4452
[root@osboxes /]# cat /proc/4452/status | grep -i swap
```

Checking SWAP usage of application | using bash scripting

- downloaded a script from net
- make a file '/opt/swap_test'
- copy paste bash script into above file.
- run `bash swap_test`
    - it will list swap usage of al apps.
- Here is a bash script that can be used here (needs no root permission)!

```
#!/bin/bash
# Get current swap usage for all running processes
# Erik Ljungstrom 27/05/2011
# Modified by Mikko Rantalainen 2012-08-09
# Pipe the output to "sort -nk3" to get sorted output
# Modified by Marc Methot 2014-09-18
# removed the need for sudo

SUM=0
OVERALL=0
for DIR in `find /proc/ -maxdepth 1 -type d -regex "^/proc/[0-9]+"`
do
    PID=`echo $DIR | cut -d / -f 3`
```

```
    PROGNAME=`ps -p $PID -o comm --no-headers`
    for SWAP in `grep VmSwap $DIR/status 2>/dev/null | awk '{ print $2 }'`
    do
        let SUM=$SUM+$SWAP
    done
    if (( $SUM > 0 )); then
        echo "PID=$PID swapped $SUM KB ($PROGNAME)"
    fi
    let OVERALL=$OVERALL+$SUM
    SUM=0
done
echo "Overall swap used: $OVERALL KB"
```

1.10.3 Process | Deamons

Here is an overview of these topics:

1.10.3.1 Process

- Process may be any task that runs over RAM.
- These process starts and then stops also by their own (if we run `ls` and then run `pwd` ; previously run `ls` will stop, etc.)

For Example : `fdisk` , `ls` , `pwd` , etc.

- Each process is assigned with a special pid that can be seen inside `/proc`
- we can also directly check pid of a specific program by:
- `pidof firefox` = pid of firefox

1.10.3.2 Deamons

- Those particular process that keeps on working unless we close them by ourselve.

For Example : `firefox`, etc

Types of Deamons :

1. Application deamons
    - no impact on system if these are killed
    - i.e., `gedit`, `firefox`, etc
2. System deamon
    - system will crash if these deamons are closed
    - i.e., `init`, etc

1.10.3.3 Playing with Services | RHEL6

Here, `INIT` (with PID=1) is the father of all processes.

- `service mysqld start` | "mysqld" is an example here
- `service httpd status` | "httpd" is an example here

- `service httpd start`
- `service httpd restart`
- `service httpd stop`
- `chkconfig httpd on` | to enable apache while booting (autostart while booting)
    - whenever our machine will boot, apache web server will start by its own due to this command.
- `chkconfig httpd off` | to disable apache while booting (autostop while booting)

1.10.3.4 Playing with Services | RHEL7_8

Here, `SYSTEMD` (with PID=1) is the father of all processes.

- `systemctl status httpd`
- `systemctl start httpd`
- `systemctl restart httpd`
- `systemctl stop httpd`
- `systemctl enable httpd` | to enable apache while booting (autostart while booting)
- `systemctl disable httpd` | to disable apache while booting (autostop while booting)

1.10 Questions

Q.no.1 : How to search PID of a program?

Q.no.2 : How to check info of CPU and memory?

Q.no.3 : What is buffer | cached?

- **Buffer memory**
    - Pre-loaded processes in RAM are called buffer
    - [*Example* : while watching a video on youtube, even if network goes at sudden, our video keeps playing for a while (till greyed heading mark), it's because, upto that mark, clip is loaded in RAM]
- **Cached memory**
    - is a temporary data stored in side hard disk (inside swap)
    - it is stored to speed up data process (for repeated data)

Q.no.4 : Write 15 examples of `ps` ?

1. List All Processes

```
[root@osboxes /]# ps -aux | less
```

2. List Processes for a Specific User

```
[root@osboxes /]# ps -u username
```

---

3. Show Process with full details

```
[root@osboxes /]# ps -elfH | less     # combination of `ps -el`, `ps -ef`, `ps -efH`
```

7. Show All Processes, Including Threads

```
[root@osboxes /]# ps -eLf
```

8. Display Process Memory Usage (RSS and VSZ)

```
[root@osboxes /]# ps -o pid,cmd,rss,vsz
```

9. Show Top N CPU Consuming Processes

```
[root@osboxes /]# ps aux --sort=-%cpu | head -n 10
```

10. List Top N Memory Consuming Processes

```
[root@osboxes /]# ps aux --sort=-%mem | head -n 10
```

11. Show Processes with a Specific Name

```
[root@osboxes /]# ps aux | grep process_name
```

12. List All Processes and Their Open Files

```
[root@osboxes /]# ps aux | xargs -n 1 lsof -p
```

13. List All Processes in current terminal

```
[root@osboxes /]# ps
```

Q.no.5 : Running a program/process in background?

In some cases, for example, when I tried to open gedit program using `gedit` in terminal.

- my that terminal stopped, because of this `gedit` program ruuning on the foreground
- my terminal remained trapped, unless I hit enterd `Ctrl+C` to stop that process.
- So, to avoid this, we can run `gedit&` to open this "gedit" in the background within the shell
- and thus, we will keep working smoothly in our very same terminal.

Q.no.6 : Differentiate between within the shell and out of the program/process in background?

within the shell (background process)	out of the shell (background process)
`gedit&`	`nohup gedit&`
Will closes when terminal closed	will keep working even after closing terminal
not closing for root user !!??	yes it keeps working (but a file "nohup.out" is created on Desktop each time) !!??

Q.no.7 : sometimes, `kill` command kills the terminal but the process that was running inside its bash keeps running?

I encountered this query!!

Even that program (firefox) was not opened using `nohup firefox&` | where program do not runs inside shell (so, can persist after closing terminal)

Q.no.8 How to check background process

We have three commands:

- `fg` = shows background process name but terminal stucks (unless `Ctrl+C`)
- `bg` = it perhaps just says, 1 jobs already running (~ if one program opened)
- `jobs` = it gives short details of background running processes

> we can simply run `ps` to see process running in current terminal.

Q.no.9 : How `kill` works? What is "SIGTERM" and "SIGKILL"? & How many total signals does `kill` have?

Working of `kill`

- when we run`kill`
- it sends direct signal to kernal to terminate to process

SIGTERM and SIGKILL

When we use this `kill PID` to terminate a program it actually gives by default signal of "15" known as **SIGTERM**
We can also write complete command `kill -15 PID` to by default kill a program (in normal way).

But, when we terminate a program forcefully, we give signal of "9", known as **SIGKILL**
So, we run `kill -9 PID`

Total signals in `kill`

`kill` command has 64 signals in total that can be seen using `kill -l`

Q.no.10 : What is "rss" and "vsz" in term of memory

RSS (Resident Set Size)

The actual amount of physical memory (RAM) actively used by a process.

VSZ (Virtual Memory Size)

The total amount of virtual memory used by a process, including both RAM and swap space.

Command to find rss and vsz

```
[root@osboxes opt]# ps -o pid,cmd,rss,vsz
```

Q.no.11 : How to mitigate the effect of FORK BOMBING ?

`:() { :|:& };:` = Fork Bomb

In Fork Bombing, at-once opening of eormous numbers of applications exuast the system and it becomes unresponsive, then and there!

To limit the effect of such attack, we can limit the maximum numbers of running processes for a specific user.

These are few steps:

- Firstly check limit of process for a user using `ulimit -u`
- In my case it was 1024 (enough to exhaust my machine)
- we will limit them to lesser numbers (i.e., 100, etc.) using `ulimit -u 100`
- Now if we run an attack, that will not work that much! its effect will be mitigated.
- But these measures are temporary.
- To make them permenant, add this limit to the end of `~/.bashrc` file of that user.
- run `source ~/.bashrc` from anywhere : OR
- from within `~` of that user, run `source .bashrc` to apply these changes without rebooting

> The above protocol will only secure a particular user for which this is done.
> If we want to apply such measure for all users of a machine, we can use `pam_limits` (according to chat.openai.com)

I tried to insert this entry of `ulimit -u 100` to limit maximum process using root user in `/etc/security/limits.conf`.

- In new terminal after this insertion, `ulimit -u` gives value of 100, in this case.
- But if I run bombing script, it hangs machine.
- Effect was not mitigated as it was in case of specific user as explained in previous example.

Q.no.12 : How to `top` output in a file, for later analysis ?

- We can don that by using redirect sign.
- To use redirect sign with it, we will run `top -n1 > top.txt`